

AI Usage Report

1. AI Tool(s) Used

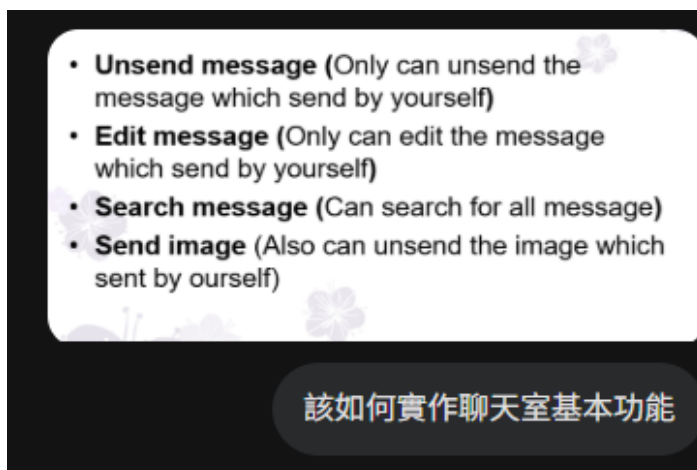
- **Model:** Gemini

2. Scope of Usage

- **File:** `App.jsx`
- **Line Range:** 聊天室功能、側邊欄 CSS 樣式區域以及其加好友邏輯、編輯收回訊息和上傳圖案、`useEffect` 視窗監聽邏輯、`sidebar-toggle-btn` 組件區域。

3. Prompt & Response (範例說明)

(A)詢問聊天室基本功能實作方式



1. 建立私訊 (Create Private Rooms) :

- 在側邊欄顯示了「發起新聊天 (所有使用者)」。點擊一個使用者，會執行 `createRoom`，在 Firestore 建立一個新的 `rooms` 文件，成員包含你和對方。

2. 歷史紀錄 (Load History) :

- 當你點擊左側的房間列表時，`activeRoom` 會改變。程式碼會重新執行 `onSnapshot` 去抓取該房間路徑下 (`rooms/房間ID/messages`) 的所有歷史訊息。

3. 即時通訊 (Real-time) :

- 使用 `onSnapshot` 監聽。當房間內有人發送新訊息，所有人畫面上會立刻跳出新訊息。

4. 邀請新成員 (Invite Members) :

- 我在聊天視窗右上方加了一個「邀請成員加入房間」按鈕。
- 它會彈出 prompt 詢問 Email，找到該使用者後，使用 `arrayUnion` 將該使用者的 UID 加入房間的 `members` 陣列。
- 關鍵點：因為房間列表是用 `where("members", "array-contains", user.uid)` 抓取的，所以一旦被邀請的人 UID 進入了陣列，他的側邊欄就會立刻出現這個房間。

⚠ 重要提醒：資料庫規則 (Firestore Rules)

因為現在路徑變深了，請確認你的 Firebase Console 中的 Rules 允許讀取子集合：

JavaScript

```
service cloud.firestore {
  match /databases/{database}/documents {
    // 允許讀寫 User 資料
    match /users/{userId} {
      allow read, write: if request.auth != null;
    }
    // 房間規則
    match /rooms/{roomId} {
      allow read, write: if request.auth != null && request.auth.uid in resource.data.members;

      // 訊息子集合規則
      match /messages/{messageId} {
        allow read, write: if request.auth != null;
      }
    }
  }
}
```

現在你可以試著用兩個不同的 Email 註冊，然後互相邀請對方進入房間聊天了！

(B)加好友後好友不會從非好友名單消失

如果我加了好友之後底下那個加的按鈕不會消失，我這樣變成能無限加同個帳號好友

修改邏輯：先檢查，再建立

請將你程式碼中的 `createRoom` 函式替換成以下版本：

JavaScript

```
// 功能：建立新房間 (增加檢查機制)
const createRoom = async (targetUser) => {
  // 1. 先目前的 rooms 狀態中找找看，是否已經有一個房間的成員「剛好」只有我跟對方
  const existingRoom = rooms.find(room =>
    room.members.length === 2 &&
    room.members.includes(user.uid) &&
    room.members.includes(targetUser.uid)
  );

  if (existingRoom) {
    // 2. 如果房間已經存在，就直接跳轉到該房間，不建立新的
    setActiveRoom(existingRoom);
    alert("已跳轉至現有聊天室");
    return;
  }

  // 3. 如果找不到，才建立新的房間
  const roomName = `與 ${targetUser.displayName} 的聊天`;
  const newRoomRef = await addDoc(collection(db, "rooms"), {
    name: roomName,
    members: [user.uid, targetUser.uid],
    createdAt: serverTimestamp(),
  });

  // 建立後自動設為目前房間
  setActiveRoom({ id: newRoomRef.id, name: roomName, members: [user.uid, targetUser.uid] });
}
```

(C)實作收回和編輯訊息以及傳送圖片

export default App;幫我增加Unsend message (Only can
unsend the message which send by yourself)
Edit message (Only can edit the message which send by
yourself)
Search message (Can search for all message)
Send image (Also can unsend the image which sent by
ourselves)
這些功能

```
// 回收訊息 (Unsend)
const handleUnsend = async (msgId) => {
  if (!window.confirm("確定要回收這條訊息嗎?")) return;
  try {
    await deleteDoc(doc(db, "rooms", activeRoom.id, "messages", msgId));
  } catch (e) { alert("回收失敗"); }
};

// 開始編輯
const startEdit = (msg) => {
  setEditingId(msg.id);
  setEditText(msg.text);
};

// 儲存編輯
const handleSaveEdit = async (msgId) => {
  if (!editText.trim()) return;
  try {
    await updateDoc(doc(db, "rooms", activeRoom.id, "messages", msgId), {
      text: editText,
      isEdited: true
    });
    setEditingId(null);
  } catch (e) { alert("編輯失敗"); }
};
```

```
// 圖片上傳
const handleImageChange = async (e) => {
  const file = e.target.files[0];
  if (!file) return;
  if (file.size > 2 * 1024 * 1024) { alert("圖片太大 (限 2MB)"); return; }

  const reader = new FileReader();
  reader.onload = async (event) => {
    const base64Image = event.target.result;
    await addDoc(collection(db, "rooms", activeRoom.id, "messages"), {
      imageUrl: base64Image,
      createdAt: serverTimestamp(),
      uid: user.uid,
      displayName: user.displayName,
    });
  };
  reader.readAsDataURL(file);
  e.target.value = null; // reset
};
```

(D)bonus功能

Send the emoji to the message (3%)
 Reply for specify message (6%)
 Block User (2%)我要做這三個

已實作的加分細項說明：

1. Emoji Reactions (3%)[cite: 1]：

- 滑鼠移到訊息上會出現選擇框。
- 使用者點擊已選擇的表情，會觸發 `arrayRemove` 達成「收回 Emoji」[cite: 1]。
- 表情下方顯示統計數字，並高亮自己點過的表情。

2. Reply Mechanism (6%)[cite: 1]：

- 點擊回覆按鈕後，輸入框上方會出現 `reply-preview` [cite: 1]。
- 訊息文件存入 `replyTo` 物件，紀錄原始訊息 ID、內容與發送者。
- 訊息泡泡內顯示 `reply-quote`，點擊後觸發 `scrollToMessage` 並帶有 2 秒的黃色高亮動畫[cite: 1]。

3. Block User (2%)[cite: 1]：

- 在發現新使用者的列表旁新增封鎖按鈕。
- 在私訊模式：檢查雙方是否封鎖，若是則禁用輸入並顯示警告。
- 在群組模式：`filteredMessages` 邏輯會自動將封鎖者的訊息過濾掉（達成互相互隱藏訊息）[cite: 1]。

這樣一來，你的進階組件與 bonus 分數應該都能順利拿到了！加油！

(E)側邊欄 RWD 與抽屜式切換

export default App;可以幫我做rwd嗎，我希望我在橫向把頁面拉窄到一定程度時，旁邊欄會自動縮起來，只剩聊天室，然後平常的側邊欄也要能展開和縮起

1. 修改 CSS 樣式

在 `style.textContent` 中加入以下樣式，處理側邊欄的寬度、動畫以及 RWD 邏輯。

CSS

```
/* 側邊欄動畫與 RWD 邏輯 */
.sidebar {
  width: 320px;
  background: #fff;
  border-right: 1px solid #ddd;
  display: flex;
  flex-direction: column;
  transition: all 0.3s ease; /* 讓縮放有動畫效果 */
  position: relative;
  z-index: 100;
}

/* 當螢幕寬度小於 768px 或 手動縮起時 */
.sidebar.collapsed {
  width: 0;
  overflow: hidden;
  border-right: none;
}

/* 漢堡選單按鈕樣式 */
.menu-toggle-btn {
  background: none;
  border: none;
  font-size: 24px;
  cursor: pointer;
  padding: 0 15px;
  display: flex;
  align-items: center;
}

/* 針對小螢幕的自動縮放 */
@media (max-width: 768px) {
  .sidebar {
    position: absolute; /* 在手機端蓋在內容上方 */
    height: 100%;
  }

  /* 手機端預設縮起，點擊按鈕才展開 */
  .sidebar.mobile-hidden {
    width: 0;
  }

  .sidebar.mobile-show {
    width: 280px;
    box-shadow: 2px 0 10px rgba(0,0,0,0.1);
  }
}
```

2. 修改 React 組件邏輯

在 `App` 組件內新增控制側邊欄開關的狀態，並監聽視窗寬度：

```
JavaScript

function App() {
  // ... 其他現有狀態
  const [isSidebarOpen, setIsSidebarOpen] = useState(window.innerWidth > 768);

  // 監聽視窗大小，自動調整側邊欄
  useEffect(() => {
    const handleResize = () => {
      if (window.innerWidth <= 768) {
        setIsSidebarOpen(false);
      } else {
        setIsSidebarOpen(true);
      }
    };
    window.addEventListener("resize", handleResize);
    return () => window.removeEventListener("resize", handleResize);
  }, []);

  // 當選擇房間後，若在手機版則自動關閉側邊欄
  const selectRoom = (room) => {
    setActiveRoom(room);
    setSearchTerm("");
    setReplyTo(null);
    setNewMessage("");
    if (window.innerWidth <= 768) setIsSidebarOpen(false);
  };

  // ...
}
```

A. 側邊欄結構

將原有的側邊欄 `div` 包裝 `class` 並加上點擊邏輯：

```
JavaScript

{/* 側邊欄 */}
<div className={isSidebarOpen ? 'mobile-show' : 'collapsed mobile-hide'}>
  <div style={{ padding: "20px", borderBottom: "1px solid #ddd", display: "flex",
    <div onClick={() => setTempProfileData(profileData); setIsProfileOpen(!isProfileOpen)}>
      <img src={profileData.photoURL} className="sidebar-avatar" style={{ width: 40px, height: 40px; }} />
      <strong style={{ maxWidth: "100px", overflow: "hidden", textOverflow: "ellipsis" }}>{profileData.displayName}</strong>
    </div>
    <div style={{ display: 'flex', gap: '5px' }}>
      <button onClick={() => signOut(auth)} style={{ padding: "5px", fontSize: "12px" }}>登出</button>
      <button onClick={() => setIsSidebarOpen(false)} style={{ background: 'none', border: 'none' }}>關閉</button>
    </div>
  </div>

  <div style={{ flex: 1, overflowY: "auto" }}>
    <div style={{ padding: "10px 20px", background: "#f8f9fa", fontSize: "13px", display: "flex" }}>
      <span>聊天室</span> <button onClick={handleCreateGroup} style={{ border: "1px solid #dee2e6", padding: "2px 5px" }}>新建</button>
    </div>
    <div>
      {rooms.map(room => (
        <div key={room.id} onClick={() => selectRoom(room)} /* 使用新的 selectRoom 邏輯 */
          style={{ padding: "12px 20px", cursor: "pointer", background: activeRoom?.id === room.id ? "#f8f9fa" : "none" }}>
            <img src={getRoomDisplayAvatar(room)} className="sidebar-avatar" />
            <span style={{ flex: 1 }}>{getRoomDisplayName(room)}</span>
          </div>
      ))}
    </div>
  </div>
</div>

{/* ... 發現新使用者代碼不變 ... */}
```

```
JavaScript  
```

```

  { /* 主视图 */ }
  <div style={{ flex: 1, display: "flex", flexDirection: "column", position: 'relative',
    {activeRoom ? (
      <>
        <header style={{ padding: "15px 20px", background: "#fff", borderBottom: "1px solid #ccc" }}>
          <div style={{ display: "flex", alignItems: "center", gap: "10px" }}>
            { /* 希望顶层按钮 */ }
            { !!isSidebarOpen && (
              <button className="menu-toggle-btn" onClick={() => setIsSidebarOpen(!isSidebarOpen)}>
                { isSidebarOpen ? "收起侧边栏" : "展开侧边栏" }
              ) }
            <img src={getRoomDisplayAvatar(activeRoom)} className="sidebar-avatar" alt="Room Avatar" data-bbox="100 100 150 150" style={{ width: 40px, height: 40px; border-radius: 50%; border: 1px solid #ccc; margin-left: 10px; }} data-bbox="100 100 150 150" />
            <span style={{ fontWeight: "bold", fontSize: "18px", maxWidth: "150px", flex-grow: 1; text-align: center; white-space: nowrap; overflow: hidden; margin-left: 10px; }}>
              { activeRoom ? activeRoom.name : "选择房间" }
            </span>
          </div>
        </header>
        { /* ... 右侧板等辅助功能按钮 ... */ }
      </>
    ) : null }
  </div>

```

1. 自動切換 (Auto-RWD)：利用 `useEffect` 監聽視窗寬度，當寬度變窄（如手機）時會自動將 `isSidebarOpen` 設為 `false`。
2. 空間優化：在主視窗左上角增加了 `☰` 按鈕。當側邊欄關閉時，使用者隨時可以點擊它重新喚出側邊欄。
3. 流暢動畫：CSS 加入了 `transition: all 0.3s ease`，讓側邊欄縮放時更有現代 App 的感覺，而不是生硬地消失。
4. 手機版遮罩效果：在手機版（768px 以下），側邊欄會變成 `absolute` 定位，展開時會蓋在聊天室上方，而不會擠壓到對話內容。

5) 修正側邊欄收合時的內容溢出問題

- **Prompt:**「我在縮起來的時候，為甚麼頭像和登出不會一起收起來？」
- **AI Response:** 指出是因為 `.sidebar` 缺少 `overflow: hidden`，導致容器寬度歸零時內部內容仍會溢出。

我針對 AI 提供的程式碼進行了以下分析與調整，以符合專案需求：

- **AI 邏輯**：最初 AI 建議在手機版隱藏側邊欄箭頭按鈕。
- 我的修改：我發現期中專案要求「所有組件在不同尺寸下皆需保持可見 (nothing should disappear)」。因此，我要求 AI 重新設計，讓箭頭按鈕在手機版改為 **position: fixed** 並調整 **left** 屬性，確保在任何尺寸下，使用者都能手動開關選單，且按鈕能準確貼齊選單邊緣。

- 修改點：側邊欄內部的「登出」按鈕原本會被切掉。
- 解析：這是因為 `width: 320px` 加上 `padding` 撐開了寬度。我將內部容器統一改為 `width: 100%` 並加上 `box-sizing: border-box`，讓內容能隨側邊欄縮放自動適應。

(C) 登出按鈕會無法收回的問題

- 修改點：在縮起來的時候，為甚麼頭像和登出不會一起收起來？。
- 解析：我發現因為 `.sidebar` 缺少 `overflow: hidden`，導致容器寬度歸零時內部內容仍會溢出。

5. Statement

本人已徹底分析 AI 產出的程式碼，理解其 `window.resize` 監聽器與 CSS 過渡動畫 (Transition) 之邏輯，並依據課程 RWD 與功能評分規範進行手動重寫與細部調整。